Committee Review v2 — full project, strict pass with remedies

Reviewer: Chair, Department of Statistics (external examiner) **Scope:** everything as of this session — JOLTEON, MEDICHAM, DITTO, CHOMP, the predictability study, the new SLOWKING implementation, and the "Poker→Pokémon" white paper. **Format:** every critique is paired with a concrete fix. Findings marked **[SCRAP]** call for rebuild, **[FIX]** for repair, **[SHIP]** are cheap and should be done now. **Overall:** materially stronger than v1 — the 0/100 engine pathology is gone, an evaluation harness now exists and has produced a real (unflattering) verdict, and SLOWKING is real tested code rather than a promise. But the project still ships probabilities that a committee would not accept, and one of tonight's new claims (that the search "is ReBeL") is not supported by the code.

A. JOLTEON — now with an empirical verdict

Critique. The harness settles it: on a temporal held-out split, JOLTEON's **log-loss (0.699) is worse than a coin flip (0.693)** and only ties player-Elo (0.691). Accuracy (54.6%) sits at the skill ceiling, but the *probabilities are overconfident* — the model is a decent ranker and a poor forecaster. This is exactly the failure a proper scoring rule is designed to catch and accuracy is designed to hide. Separately, the additive form still cannot represent a non-transitive metagame.

Fix. Three, in order of cost:

1. **[SHIP → DONE] Recalibrate.** Fit a single temperature `T>1` (Platt scaling) on a calibration slice of train and divide the logits by it. **Result (`engine/calibrate.py`): T = 5.96** — the model's logits were ~6× too large, i.e. severely overconfident. Held-out log-loss **0.705 → 0.692**, which now **beats the coin (0.693)** and ties Elo (0.691); Brier 0.255 → 0.250. Honest caveat: T≈6 means calibration works by shrinking predictions hard toward 0.5 — it removes false confidence but cannot create signal. The team sheet is worth only a sliver over a coin; real lift needs fix #2.
2. **[FIX] Add a non-transitive term.** Low-rank antisymmetric interaction `⟨u(A),v(B)⟩−⟨u(B),v(A)⟩` so the model can encode rock-paper-scissors matchups the sum-of-abilities cannot.
3. **[SHIP] Show the truth on the site.** Display JOLTEON's held-out log-loss vs. baselines. A model that ties Elo should say so, not imply oracular power.

B. MEDICHAM — the rollout

Critique. (i) Policy-bounded: a behavior-cloned policy has covariate shift, and yours over-credits speed-control cores because the clone can't disrupt. (ii) The damage engine has **never been validated against the reference the Smogon damage calculator** — "grounded" is unearned until it is. (iii) You show a point estimate; 400 rollouts carry SE ≈ ±2.5%.

Fix. [SHIP] Print the Monte-Carlo CI (`±1.96·√(p(1−p)/N)`) next to every rollout %. **[FIX]** Write a validation script that samples 100 attacker/defender/move triples, compares your `dmgRange` to the Smogon damage calculator, and reports max/mean error; fix rounding until it matches. **[FIX→SCRAP]** DAgger-correct the policy against the real engine, and make the *real* Showdown engine the scorer — at which point the hand-rolled browser engine becomes a labeled approximation, not the source of truth.

C. DITTO — still mis-named, but now fixable in-repo

Critique. Unchanged from v1: coordinate ascent against a static usage gauntlet, scored by a model (JOLTEON) that Section A shows barely beats a coin — squared. It is not a double oracle.

Fix. [SCRAP & REBUILD] You now own the machinery to do it right: `slowking/nash.py` + `solver.team_preview`. Rebuild DITTO as a genuine meta-game — maintain a team population, fill an empirical payoff matrix with **real-engine** rollouts, solve the **meta-Nash**, best-respond to the equilibrium mixture, iterate (PSRO/double-oracle). Output the equilibrium *mixture of teams*, because in a cyclic meta there is no single best team. Until then, at minimum score with the real engine and stop printing "double-oracle."

D. SLOWKING — the new code, held to the same standard

Credit first: `nash.py`, `belief.py`, `ismcts.py`, `solver.py` are implemented and unit-tested; the equilibrium solving is correct. Now the strict part.

D1 — [SCRAP the claim, keep the code]. The search is PIMC/IS-MCTS, not ReBeL. `ismcts.py` samples one hidden world per iteration (root determinization) and searches it as if fully observed. That is Perfect-Information Monte Carlo, which provably suffers **strategy fusion** (it assumes it will later know hidden information it will not) and **non-locality** (Long, Sturtevant, Buro, Furtak 2010). ReBeL exists *specifically to avoid this* by searching over the public belief state itself. So the white paper's "the architecture is ReBeL" is not supported by the current implementation — the code is a documented rung below it.

- **Fix.** Two honest paths: (a) relabel the current engine as an **IS-MCTS/PIMC baseline** with known limits (accurate, cheap), and/or (b) implement true PBS re-solving — CFR over the belief distribution with a value function at the leaf — which *is* ReBeL. An interim middle option is **αμ**, which repairs PIMC's strategy fusion at modest cost. The white paper must be edited to match whichever you ship.

D2 — [FIX]. The verified search does not scale. The counterfactual update computes a rollout *per action, per node* — O(branching) — which is why it's only verified on 2- and 3-action games. Champions has hundreds of joint actions per turn.

- **Fix.** Switch to **outcome-sampling MCCFR** (sample one action, importance-weight the regret) or lean on the value network so you never enumerate actions at depth. Keep the exact version only as the correctness oracle on toy games.

D3 — [FIX]. `belief.py` **assumes independent brings.** The 15-way bring distribution is a product of per-mon in/out rates. Real brings are **correlated** — cores travel together (rain setter ⇒ swimmer). Independence misestimates the distribution the whole search integrates over.

- **Fix.** Learn the joint bring distribution from ladder `brought` data (or at least condition on revealed cores), not an independence product.

D4 — [FIX]. The team-preview Nash solves a matrix of noisy estimates. Each payoff cell is an MC win rate with real variance; regret matching can chase that noise, and the reported equilibrium value has an unstated CI.

- **Fix.** Budget enough rollouts per cell (or use common random numbers across cells), and report the value with a bootstrap interval plus the solved exploitability.

D5 — [honest status]. No value net, no self-play training. The leaf is a slow rollout and nothing is learned yet. This is a correct **chassis**, not a bot — the white paper says as much, which is good; keep it that way.

E. The white paper

Critique. Excellent framing (the wide-and-shallow observation is a real contribution), but it **overclaims ReBeL** per D1, and it asserts the value net will be "easier than poker's" without evidence — plausible from the short horizon, but state it as a hypothesis.

Fix. [SHIP] Edit §2/§5 to say ReBeL is the *target*; the current engine is IS-MCTS with documented strategy-fusion limits, to be repaired via αμ or PBS re-solving. Mark the "easier value net" as a conjecture to be tested.

F. Predictability study

Critique. Unchanged: matching two aggregate win rates is not validation (base-rate fallacy).

Fix. [FIX] You now have `eval_harness.py` (per-game log-loss/Brier/calibration). Redo the study as a proper skill/luck decomposition with intervals, à la Lopez et al., and drop the "we predict at 55% so we're right" framing.

G. CHOMP

Critique. Greedy coverage max against a static opponent; bring/lead is a simultaneous game.

Fix. [FIX] Feed the bring payoff matrix to `nash.py` and return the minimax bring mix — the same solver DITTO and SLOWKING now use.

H. Cross-cutting (the thing that most threatens the whole project)

Critique. The website still renders probabilities as exact points, with no CI, no calibration statement, and no baseline context — even though we now *know* JOLTEON ties a coin in log-loss.

Fix. [SHIP] Every number on the site gets a CI or an honesty caption. Put the eval verdict where users can see it. Credibility comes from stating your error bars, not hiding them.

What improved since v1 (fair credit)

- The 0/100 MEDICHAM pathology is gone (real doubles engine, embedded, mirror 0.50).
- An evaluation harness exists and produced a real, unflattering verdict — the single most important addition.
- SLOWKING is tested code, and `nash.py` gives DITTO and CHOMP a real equilibrium tool they were missing.

Prioritized remediation roadmap

1. **[tonight] Recalibrate JOLTEON** (temperature scaling) and re-run the harness — *done below*.

2. **[tonight] Edit the white paper** to stop claiming ReBeL; call the search IS-MCTS/PIMC with known limits.
3. **Validate the damage engine** against the Smogon damage calculator; add MC confidence intervals to MEDICHAM and DITTO outputs.
4. **Rebuild DITTO** on `nash.py` as a real meta-Nash solve; rebuild CHOMP's bring as minimax.
5. **Wire the engine adapter** (`sim/`), then replace SLOWKING's PIMC with outcome-sampling MCCFR / PBS re-solving and train the value net on self-play.
6. **Redo the predictability study** with proper scoring; surface CIs and the eval verdict on the site.

The project has crossed from "impressive demo" to "honest instrument with known holes." That is real progress. Close the holes in the order above and it becomes defensible.

Sources

- Understanding the Success of PIMC Sampling (strategy fusion, non-locality)
- ReBeL — RL+Search over public belief states · DeepStack · Libratus
- Brier score · Proper scoring rules
- PSRO / double oracle · How often does the best team win?